What Enables In-Context Learning in Transformers? An Analysis of Attention Mechanisms

Anish Kasam akasam@ucsd.edu
Dhanvi Patel dhp003@ucsd.edu
Krish Prasad krprasad@ucsd.edu
Shou Tai Yue syue@ucsd.edu
Arya Mazumdar amazumdar@ucsd.edu

Abstract

In-context learning (ICL) refers to the ability of transformer models to adapt to new tasks at inference time using only examples provided in the prompt, without updating model parameters. Recent theoretical work suggests that this capability may arise because transformers implicitly implement optimization procedures, such as gradient descent, within their forward pass.

Prior work has shown that linear self-attention (LSA) can exactly replicate a single gradient descent step for linear regression tasks, providing strong evidence for this optimization-based interpretation of ICL Oswald et al. (2023). However, LSA is a simplified attention mechanism and does not fully represent the architectures used in modern transformer systems.

In this work, we investigate whether optimization-like in-context learning behavior persists across a broader range of attention mechanisms. We train transformer models with several attention variants, including standard softmax attention, linear attention, gated linear attention, grouped-query attention, sparse attention, and low-rank attention on synthetic linear regression tasks under identical experimental settings. We evaluate each model across multiple experimental conditions by measuring prediction error and cosine similarity with analytic gradient descent updates.

Our results show that attention mechanisms that preserve dense and largely linear token interactions, such as linear self-attention and kernelized linear attention, most closely reproduce gradient-descent-like behavior. In contrast, architectures that impose strong structural constraints, such as sparse attention, fail to converge to the same optimization dynamics. These findings suggest that the structural properties of attention mechanisms play a central role in enabling optimization-like in-context learning.

Website: https://dhp003.github.io/dsc180b-website/
Code Repo:
https://github.com/Shou-Yue/Incontext-Learning-of-Attention-Mechanisms

1 Introduction 3
2 Methods 4
3 Results 9
4 Discussion 12
5 Conclusion 13
Appendices A1

1 Introduction

Transformer architectures are the underlying architecture behind many modern machine learning systems (LLMs, image/video generation models). One of the key capabilities that distinguishes transformers from traditional models is their ability to learn in-context, a phenomenon where a model can adapt to a new task using just the examples provided in the prompt at inference time. This adaptation occurs without updating model parameters, suggesting that the learning procedure is already implemented internally during the forward pass.

Recent theoretical work argues that ICL may arise from transformers implicitly performing optimization procedures such as gradient descent over the context examples. Rather than memorizing mappings, the model may learn to approximate an update rule that extracts task structure from the prompt. Additionally, prior research has shown that linear self-attention (LSA), under certain assumptions, can exactly replicate a single gradient descent step for linear regression with mean squared error loss Oswald et al. (2023). Empirical evidence further demonstrates that trained transformers can match classical regression solutions on synthetic tasks, reinforcing this optimization-based interpretation.

In Quarter 1, we tested this hypothesis by training transformers on synthetic linear regression tasks across underparameterized, overparameterized, and sparse regimes. We observed strong alignment between model predictions and analytic solutions such as least squares and LASSO. We also confirmed the equivalence between LSA and a one-step gradient descent update, and observed similar behavior in pretrained language models when prompted with regression examples.

However, most prior analyses focus primarily on linear self-attention (LSA), which is not fully representative of the attention mechanisms used in modern transformer systems. Real-world architectures rely on softmax-based dense attention, sparsity constraints, gating mechanisms, low-rank approximations, and grouped-query attention to improve computational efficiency and scalability. These architectural modifications introduce structural constraints and nonlinearities, raising an important question: is optimization-like ICL a general property of attention mechanisms, or does it depend on specific architectural assumptions?

This motivates the central research question of our Quarter 2 project:

To what extent do different attention mechanisms support optimization like in context learning, and which architectural features are essential for this behavior?

To systematically analyze the role of attention mechanisms in enabling in-context learning, we design three controlled experiments. First, we study how optimization-like behavior emerges during training by sweeping the number of training steps. Second, we test whether transformer depth corresponds to multiple implicit optimization steps by varying the number of layers. Third, we analyze how performance scales with the number of in-context examples. Across all experiments, we measure prediction error and alignment with gradient descent to determine whether different attention mechanisms implement optimization-like update rules.

2 Methods

2.1 Synthetic Linear Regression Tasks

To evaluate in-context learning behavior, we train models on synthetic linear regression tasks. The same synthetic data generation process is used for all models. Each task is defined by a randomly sampled weight matrix $W^\star$ that maps inputs x to outputs y

$$y = W^\star x$$

where:

- $x \in \mathbb{R}^d$ is drawn from a zero-mean distribution
- $W^\star \in \mathbb{R}^{m \times d}$ is sampled per task
- $y \in \mathbb{R}^m$ is the target output

For each task, we generate:

- N context pairs (x_i, y_i),
- one query input x^* with target y^*.

The model receives only the context examples and the query input and must predict y^*. A new regression task (i.e., new $W^\star$) is sampled for every training example, ensuring the model must perform true in-context learning rather than memorization.

2.2 Token Construction

Following the constructed-token setup in the provided codebase, each input sequence is built by concatenating inputs and outputs within the same token.

Each context token contains both the input and output:

$$\text{context token}_i = [x_i \,||\, y_i]$$

$$\text{query token} = [x^* \,||\, 0]$$

where $||$ denotes concatenation and the query output portion is initialized to zero. The model is trained to predict the missing y^* component.

This representation allows attention mechanisms to operate directly over joint input–output tokens and enables interpretation of the model as implicitly performing an optimization procedure.

2.3 Attention Mechanisms Evaluated

We evaluate several attention variants implemented within a transformer. In all cases other than the layer sweep setup, we utilize a single head, eight layer Transformer to isolate the mechanism's behavior and maintain a direct comparison to the theoretical 1-step gradient descent baseline.

2.3.1 Standard Softmax Attention

The baseline model utilizes the standard scaled dot-product attention:

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

The softmax nonlinearity introduces a competitive normalization across context examples, which we hypothesize may either sharpen the model's focus or hinder its ability to perform purely linear weight updates.

2.3.2 Linear Self-Attention (LSA)

Following prior theoretical work, the LSA model removes the softmax activation and the scaling factor:

$$\text{Attn}(Q, K, V) = (QK^\top)V$$

LSA is of particular interest as it is mathematically capable of exactly replicating a single step of gradient descent on a mean-squared-error loss.

2.3.3 Gated Linear Attention (GLA)

GLA introduces a data-dependent gating mechanism that allows the model to dynamically weight information. The recurrence can be viewed as:

$$O_t = g_t \odot (q_t k_t^\top) + (1 - g_t) \odot O_{t-1}$$

where g_t is a forget gate. This mechanism may implement a form of preconditioned gradient descent, offering more flexibility than standard LSA Li et al. (2025).

2.3.4 Grouped-Query Attention (GQA)

GQA serves as an interpolation between Multi-Head Attention (MHA) and Multi-Query Attention (MQA). It divides H query heads into G groups, where each group shares a single key-value pair:

$$\mathrm{GQA}(Q,K,V) = \mathrm{concat}(\mathrm{head}_1,\ldots,\mathrm{head}_H)W^O$$

where for each head i in group g:

$$\mathrm{head}_i = \mathrm{Attn}(Q_i, K_g, V_g)$$

GQA reduces computational cost by sharing key and value projections across multiple query heads. We evaluate if this reduction in KV-head resolution affects the precision of the implicit regression solution compared to full MHA.

2.3.5 Sparse Attention

Sparse attention restricts each query token to attend only to a subset of context tokens, rather than the full sequence. For a query at position i, let S_i denote the allowed attention set. Then

$$\mathrm{SparseAttn}(q_i,K,V) = \sum_{j\in S_i} \alpha_{ij} v_j, \qquad \alpha_{ij} = \frac{\exp\left(q_i^\top k_j/\sqrt{d_k}\right)}{\sum_{\ell\in S_i}\exp\left(q_i^\top k_\ell/\sqrt{d_k}\right)}.$$

We use this variant to test ICL robustness under restricted context and to assess whether global context is necessary for stable weight inference.

2.3.6 Low-Rank Attention (Linformer-style)

Low-rank variants approximate the $N\times N$ attention matrix by projecting the Keys and Values into a lower-dimensional space $k \ll N$ using projection matrices E and F:

$$\mathrm{Attn}(Q,EK,FV) = \mathrm{softmax}\left(\frac{Q(EK)^\top}{\sqrt{d_k}}\right)FV$$

This tests whether a compressed summary of the context is sufficient for performing regression.

2.3.7 Kernelized Linear Attention

Kernelized linear attention approximates softmax attention by applying feature map transformations to queries and keys before computing attention scores. The attention operation can be written as

$$\mathrm{Attn}(Q,K,V) = \phi(Q)\left(\phi(K)^\top V\right),$$

where $\phi(\cdot)$ is a feature map that approximates the softmax kernel. This formulation allows attention to be computed in linear time with respect to sequence length while still enabling dense interactions between tokens.

Because kernelized linear attention preserves global information flow across the sequence, it provides a useful test of whether approximating softmax attention while maintaining dense token interactions is sufficient to support optimization-like in-context learning.

2.4 Training Procedure

All models are trained using mean squared error loss on the query prediction.

- Mean squared error (MSE) loss on the query prediction,
- Large batch sampling of independently generated regression tasks.

At each training step:

1. A batch of regression tasks is sampled.
2. Context and query tokens are constructed.
3. The transformer processes the sequence.
4. The predicted output is compared to the ground-truth value y^*.
5. Backpropagate gradients and update parameters.

Evaluation is performed on a set of regression tasks generated using the same procedure.

2.5 Experimental Design

2.5.1 Training-Step Sweep

To evaluate whether different attention mechanisms learn optimization-like behavior during training, we vary the number of training steps while keeping all other parameters fixed.

Training steps range from 0 to 100k. At each checkpoint we evaluate the model on held-out regression tasks and measure both prediction error and cosine similarity between the model's predicted update and the analytic gradient descent update.

This experiment tests whether optimization-like behavior emerges gradually during training and whether certain attention mechanisms converge toward gradient-descent-like solutions more reliably than others.

2.5.2 Layer Sweep

Prior theoretical work suggests that transformer depth may correspond to multiple implicit optimization steps. To test this hypothesis, we vary the number of transformer layers while holding training steps and context length constant.

We evaluate models with 2, 4, 8, 16, 32, and 64 layers. If layers correspond to iterative

optimization updates, deeper models should exhibit stronger alignment with multi-step gradient descent solutions.

2.5.3 In-Context Example Sweep

To analyze how in-context learning scales with the number of examples provided in the prompt, we vary the number of context pairs presented to the model during inference Garg et al. (2023).

The number of context examples is varied while keeping model depth and training steps fixed. This experiment evaluates whether additional examples improve the model's ability to infer the underlying regression weights and whether attention mechanisms differ in how efficiently they utilize context information.

2.6 Standardized Experimental Settings

All attention mechanisms are evaluated under identical settings to ensure that differences in performance arise from the attention architecture rather than confounding factors. Although experiments have different parameters depending on the evaluated feature, the standard sizes are listed below

- Input dimension: 20
- Context length: 41
- Number of attention heads: 1
- Number of attention layers: 2-16, default = 8
- Curriculum training schedule applied consistently across all models: 0-20000

These standardized settings allow direct comparison across attention variants while isolating the effect of each experimental variable.

2.7 Evaluation Metrics

To evaluate whether the model behaves like an optimizer, we measure:

- **Test Loss**: Final MSE on unseen tasks.
- **Alignment with Gradient Descent**: Cosine similarity between Transformer update directions and true gradient descent updates.
- **Functional Interpolation**: Performance when interpolating predictions between the Transformer and explicit GD solutions.
- **Attention Structure**: Visualization of attention weights to inspect query-to-context information flow.

All attention variants are evaluated under identical data distributions and training conditions to isolate the effect of the attention mechanism itself.

3 Results

3.1 Training-Step Sweep

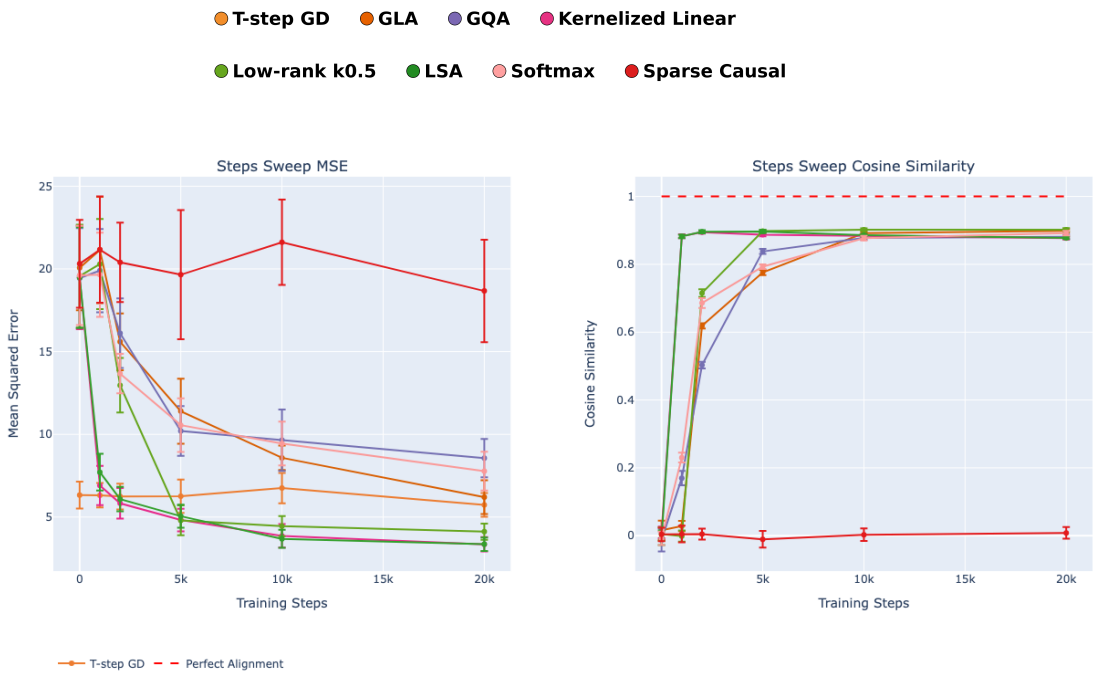

Figure 1: Training-step sweep across attention mechanisms.

Top: legend indicating the attention variants.

Bottom: test mean squared error (MSE) and cosine similarity between model predictions and the analytic one-step gradient descent update as training progresses. Mechanisms that preserve dense linear interactions converge more rapidly to gradient-descent-like behavior.

To evaluate whether different attention mechanisms learn optimization-like behavior during training, we varied the number of training steps while keeping the model architecture and context length fixed. Figure 1 shows the test mean squared error (MSE) and cosine similarity between model predictions and the one-step gradient descent solution as training progresses.

Most attention mechanisms show clear improvement as training steps increase. Test error decreases and cosine similarity to the gradient descent update increases over time. Kernelized linear attention and linear self-attention (LSA) converge the fastest, reaching low MSE and high cosine similarity within relatively few training iterations.

Grouped-query attention (GQA), gated linear attention (GLA), and standard softmax attention also improve steadily with additional training steps. These models gradually approach high cosine similarity to the gradient descent solution and achieve similar final performance levels.

Sparse causal attention behaves differently from the other mechanisms. Across all training steps, it maintains relatively high test error and its cosine similarity to the gradient descent update remains close to zero, indicating that it does not converge under the same training conditions.

3.2 Layer Sweep

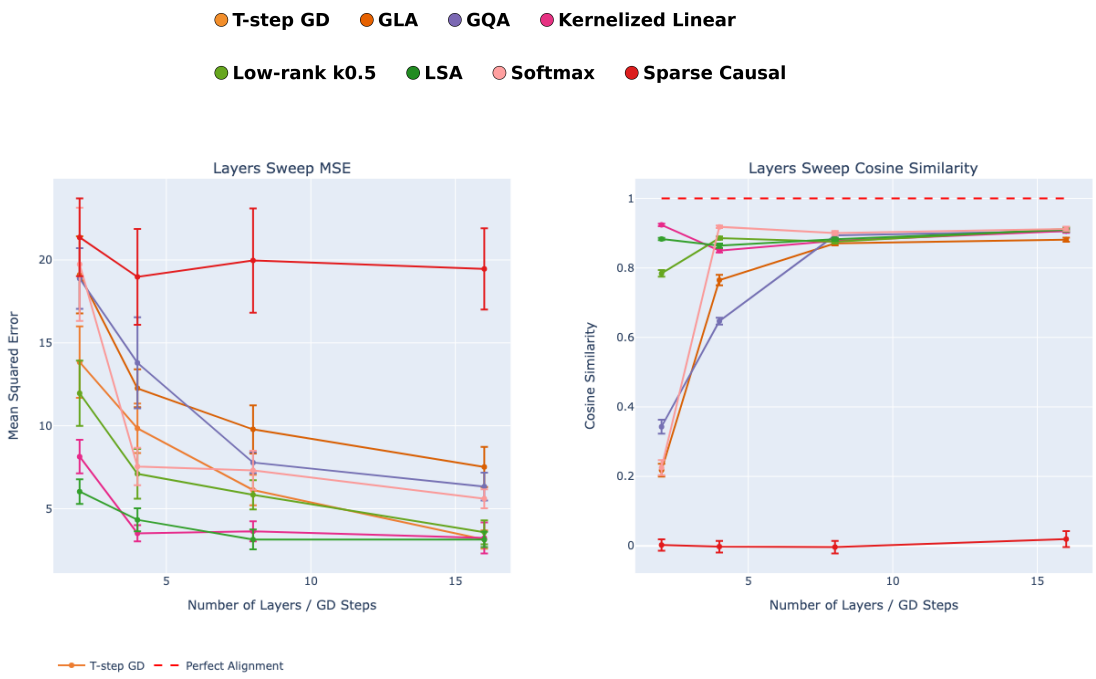

Figure 2: Layer sweep across attention mechanisms.
Top: legend indicating the attention variants.
Bottom: test mean squared error (MSE) and cosine similarity between model predictions and the analytic one-step gradient descent update as the number of transformer layers increases. Across most mechanisms, deeper models exhibit lower prediction error and stronger alignment with gradient descent, suggesting that transformer depth may correspond to multiple implicit optimization steps.

To investigate the relationship between network depth and optimization-like behavior, we varied the number of transformer layers while keeping the training steps and context length fixed. Figure 2 shows test mean squared error (MSE) and cosine similarity between model predictions and the one-step gradient descent solution as the number of layers increases.

Across most attention mechanisms, increasing the number of layers leads to lower test error and stronger alignment with the gradient descent update. Kernelized linear attention and linear self-attention (LSA) improve the fastest, reaching low MSE and high cosine similarity with relatively few layers.

Grouped-query attention (GQA), gated linear attention (GLA), and standard softmax attention also show steady improvement as depth increases. These models gradually approach high cosine similarity to the gradient descent solution and achieve similar performance levels at larger layer counts.

Sparse causal attention again behaves differently from the other mechanisms. Its test error remains high across all layer counts, and the cosine similarity to the gradient descent update stays near zero. This indicates that increasing depth alone does not lead to convergence for the sparse attention model under the same experimental conditions.

3.3 In-Context Example Sweep

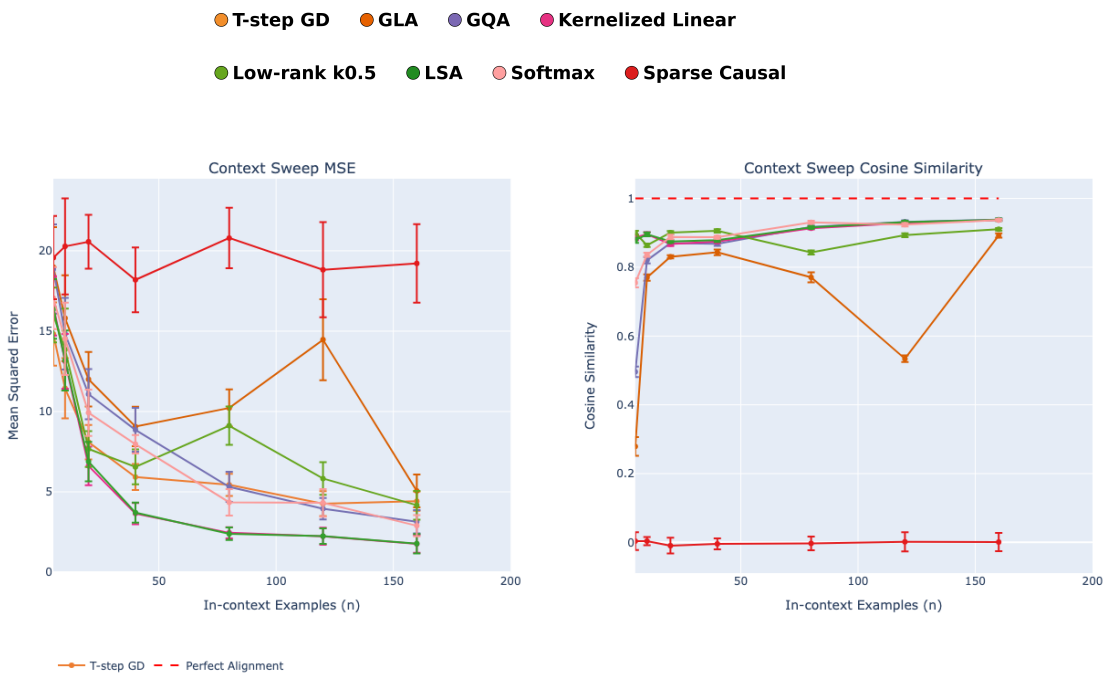

Figure 3: In-context example sweep across attention mechanisms.
Top: legend indicating the attention variants.
Bottom: test mean squared error (MSE) and cosine similarity between model predictions and the analytic one-step gradient descent update as the number of context examples increases. Performance improves as more examples are provided in the prompt, indicating that most attention mechanisms can leverage additional context to better infer the underlying regression task.

To examine how performance scales with the number of in-context examples, we varied the context length while keeping the number of training steps and layers fixed. Figure 3 shows test mean squared error (MSE) and cosine similarity between model predictions and the one-step gradient descent solution as the number of in-context examples increases.

Across most attention mechanisms, increasing the number of examples in the context leads to lower test error and stronger alignment with the gradient descent update. Kernelized linear attention and linear self-attention (LSA) again improve rapidly, achieving low MSE and high cosine similarity as the number of examples increases.

Grouped-query attention (GQA), gated linear attention (GLA), and standard softmax attention also benefit from additional context. These models show steady reductions in test error and approach high cosine similarity to the gradient descent solution at larger context sizes.

Sparse causal attention exhibits a different pattern. Its test error remains relatively high as the number of in-context examples increases, and the cosine similarity to the gradient descent solution remains near zero across all tested context lengths. This suggests that increasing context alone does not improve alignment with the gradient descent update for this mechanism under the same experimental conditions.

4 Discussion

Across all experiments, two mechanisms consistently demonstrated the fastest convergence and strongest alignment with the target solution: Kernelized Linear Attention and Linear Self-Attention (LSA). Both mechanisms rapidly reduced mean squared error and achieved high cosine similarity, closely matching the behavior expected from iterative gradient descent. This performance is likely due to the fact that both mechanisms preserve the core linear structure of the attention operation while avoiding nonlinear transformations that could distort the underlying update rule.

In particular, LSA directly computes $(QK^\top)V$ without the softmax normalization present in standard attention. Prior theoretical work has shown that this formulation can exactly replicate a single gradient descent step on a mean squared error objective under appropriate parameterization. Because the experiments in this project are designed to emulate gradient-based optimization in context, LSA naturally aligns with the structure of the task. As a result, increasing the number of layers effectively corresponds to performing additional gradient descent iterations, explaining the steady improvement in both MSE and cosine similarity as depth increases.

Kernelized Linear Attention shows similar behavior despite using a different formulation. Kernelized approaches approximate attention using feature maps that transform queries and keys before computing attention scores. While this changes the implementation of the attention computation, the resulting operation still preserves a largely linear interaction between tokens. In practice, this allows the mechanism to approximate the same iterative update process that underlies gradient descent, which likely explains why it converges quickly and achieves strong alignment across both the layer sweep and context sweep experiments.

Other mechanisms, including GQA and the low-rank attention variants, also demonstrate the ability to converge toward the target solution, though typically with slower convergence

or slightly higher final error. These mechanisms retain the essential pairwise interaction structure of attention but introduce structural constraints such as reduced dimensionality or grouped representations. While these constraints reduce computational cost, they also limit the expressiveness of the attention matrix, which may explain the somewhat slower alignment with the optimal gradient descent direction. Nonetheless, the fact that these mechanisms still achieve high cosine similarity suggests that the core optimization behavior remains recoverable even under moderate architectural constraints.

In contrast, sparse causal attention consistently fails to converge and maintains very low cosine similarity across experiments. A likely explanation is that the sparse pattern restricts each token's receptive field, preventing the model from accessing the full set of in-context examples when computing updates. The optimization process being emulated requires aggregating information across all prior examples to approximate the gradient of the loss function. When the attention pattern is sparsified, the model loses access to portions of this information, making it difficult to compute an accurate update direction. As a result, the model cannot effectively approximate the gradient descent dynamics that the other mechanisms capture.

Overall, these results suggest that the ability to emulate gradient-based optimization in context depends strongly on preserving dense, globally accessible interactions between tokens. Mechanisms that maintain this structure while remaining close to the linear attention formulation appear particularly well suited for in-context optimization tasks, whereas architectures that significantly restrict token interactions may struggle to reproduce the necessary update dynamics.

5 Conclusion

5.1 Limitations

While several attention variants exhibited behavior consistent with optimization-like in-context learning, our conclusions remain fundamentally empirical. In particular, although we observed strong alignment between certain attention mechanisms and one-step gradient descent (and in some cases its variants), these results are based solely on numerical experiments. We do not provide formal mathematical proofs demonstrating that these architectures provably implement gradient descent or any specific optimization algorithm.

Even in cases where cosine similarity to gradient descent updates was high, this only establishes behavioral resemblance under the specific task distribution studied (synthetic linear regression with mean squared error). It does not imply that the internal computation is strictly equivalent to gradient descent, nor does it guarantee that such behavior generalizes beyond the controlled experimental setting.

Additionally, our experiments focus exclusively on linear regression tasks with carefully constructed token representations. This setup is intentionally simplified to allow theoretical interpretability, but it does not capture the complexity of real-world tasks where nonlinear-

ities, distribution shifts, and structured data may significantly alter model behavior. As a result, conclusions drawn here may not directly transfer to large-scale pretrained language models or more complex learning scenarios.

Another limitation is that we evaluate only single head transformer architectures to isolate attention behavior in both the training step and in context example sweep. Although this improves interpretability, modern transformers rely on multiple heads and increased layers, residual connections, normalization layers, and nonlinear feedforward networks. These additional components may meaningfully interact with attention mechanisms and alter optimization-like dynamics in ways not captured by our study.

Finally, although some attention variants appear to diverge from strict gradient descent behavior, we do not provide a theoretical framework explaining why these divergences occur. Without analytical grounding, the trends we observe should be interpreted as descriptive rather than explanatory.

5.2 Future Research

A natural next step is to develop formal mathematical analyses of the attention variants studied in this work. While prior results establish that linear self-attention can replicate a single gradient descent step under specific assumptions, similar analytical characterizations do not yet exist for many practical attention mechanisms, including softmax attention, gated linear attention, grouped-query attention, sparse attention, and low-rank approximations.

Future work could aim to:

- Derive explicit conditions under which softmax-based attention approximates gradient descent or a related optimization procedure.
- Formally characterize gated linear attention as implementing a variant of preconditioned or momentum-based gradient descent, and identify the implicit preconditioning matrix induced by the gating mechanism.
- Analyze how low-rank projections constrain the space of achievable update directions and whether they correspond to truncated or projected gradient steps.
- Study sparse attention patterns as structured approximations to full-batch gradient updates, potentially corresponding to coordinate descent or block-gradient methods.

More broadly, constructing explicit correspondences between architectural constraints and known optimization algorithms may clarify why certain attention mechanisms exhibit stronger alignment with gradient descent than others. Rather than relying solely on cosine similarity metrics, future research could attempt to reverse-engineer the exact implicit objective function and update rule implemented by each mechanism.

Extending analysis beyond single-step updates is also important. Multi-layer transformers may implement iterative optimization procedures across layers, analogous to multiple gradient steps. Formalizing this perspective could provide insight into depth as a proxy for optimization time.

Finally, proving lower bounds or impossibility results for certain attention structures would

be equally valuable. If specific architectural constraints prevent exact implementation of gradient descent, understanding these limitations could guide the design of more efficient architectures that preserve optimization-like in-context learning.

Developing rigorous mathematical proofs and constructive analyses of gradient-descent-like behavior across attention variants would move the field from empirical observation toward theoretical understanding, providing a principled explanation for the trends observed in this study.

References

Garg, Shivam, Dimitris Tsipras, Percy Liang, and Gregory Valiant. 2023. "What Can Transformers Learn In-Context? A Case Study of Simple Function Classes." [Link]

Li, Yingcong, Davoud Ataee Tarzanagh, Ankit Singh Rawat, Maryam Fazel, and Samet Oymak. 2025. "Gating is Weighting: Understanding Gated Linear Attention through In-context Learning." [Link]

von Oswald, Johannes, Eyvind Niklasson, Ettore Randazzo, João Sacramento, Alexander Mordvintsev, Andrey Zhmoginov, and Max Vladymyrov. 2023. "Transformers learn in-context by gradient descent." [Link]

Appendices

A.1 Gradient Descent Baseline A1
A.2 Cosine Similarity Metric A1
A.3 Attention Mechanisms Evaluated A2
A.4 Reproducibility A2

A.1 Gradient Descent Baseline

To evaluate whether transformer models behave like optimization procedures, we compare model predictions with the analytic one-step gradient descent solution for linear regression.

Given context examples $\{(x_i, y_i)\}_{i=1}^{N}$, the mean squared error loss is defined as

$$L(W) = \frac{1}{2N} \sum_{i=1}^{N} \|W x_i - y_i\|^2.$$

A single step of gradient descent with learning rate η produces the update

$$W' = W - \eta \nabla_W L(W).$$

Predictions from this updated weight matrix provide the gradient descent baseline used for comparison with transformer predictions.

A.2 Cosine Similarity Metric

To quantify alignment between transformer predictions and gradient descent, we compute the cosine similarity between the update direction predicted by the transformer and the analytic gradient descent update.

Let Δy_{model} denote the update predicted by the transformer and Δy_{GD} denote the analytic gradient descent update. The cosine similarity is defined as

$$\text{cosine similarity} = \frac{\Delta y_{\text{model}} \cdot \Delta y_{\text{GD}}}{\|\Delta y_{\text{model}}\| \|\Delta y_{\text{GD}}\|}.$$

Values close to 1 indicate strong alignment between the transformer's implicit update rule and the gradient descent update.

A.3 Attention Mechanisms Evaluated

The following attention mechanisms were evaluated in our experiments.

- **Softmax Attention**: Standard scaled dot-product attention used in transformers.
- **Linear Self-Attention (LSA)**: Removes the softmax operation, allowing attention to be computed as $(QK^\top)V$.
- **Kernelized Linear Attention**: Applies feature map transformations to queries and keys to approximate attention.
- **Gated Linear Attention (GLA)**: Introduces gating mechanisms to control the flow of information across tokens.
- **Grouped Query Attention (GQA)**: Shares key-value projections across groups of query heads to improve computational efficiency.
- **Sparse Causal Attention**: Restricts attention to a limited subset of tokens.

All mechanisms were evaluated under identical training conditions.

A.4 Reproducibility

Models were trained using identical datasets and training procedures across all attention mechanisms to ensure a controlled comparison.

The code used to generate experiments and plots is available in the project repository linked in the main report. Random seeds were fixed during evaluation to ensure consistent comparisons between models.